

Team Contract

Hunter Harris, Alayna Fields, Aidan Feess, Grayson Lessard

1. How our team will work together
 - a. Communication
 - i. How fast do we want someone to respond? What do we do if they take too long?
 1. Within 3 hours
 2. If someone dies (one of our team members dies), within a few days (you are expected to answer from grave)
 - ii. What communication channel will we use?
 1. Discord
 - iii. What kind of messages go on the communication channel?
 1. Anything relevant to the project to keep it all in once place
 - b. Settling disagreements
 - i. How will we categorize the type of disagreement we are having?
 1. We can label the disagreement based on if it's technical, personal, requirements, or a process.
 - ii. Who decides and how when members cannot agree?
 1. Members that are involved with disagreement would be the deciding factor of resolving conflict
 - c. Missed expectations
 - i. What will the team do when a member does not follow one of the rules in this contract?
 1. There will be a punishment decided by the other team members
2. How code joins the codebase
 - a. Getting a change in
 - i. List the steps a change must go through before it becomes part of your shared code. Name the point at which the change stops belonging only to its author.
 1. Exists on branch
 2. Reviewed by at least one other person
 3. Tested by at least one other person
 - b. Review authority
 - i. Who can stop a change, and on what grounds?
 1. The reviewer who reviews it can stop the change if they disagree with the implementation and/or it does not work. It should then be reviewed by another person before it continues.
 - c. Author-reviewer disagreements
 - i. When do an author and a reviewer settle a disagreement themselves, and when does it go to your rule in item 2?
 1. If they can come up with a solution within a couple hours or there is a definitive fix, it should be solved between author and reviewer.

3. Hypothesis to Test

a. Roles on a task

- i. Who is the DO'er and Approver? May one person hold both? Recording of consultants?
 - 1. One person shall be the task holder and one person of the group shall be the task reviewer (dependent on task)
 - 2. There shouldn't be a need for this situation, if the prime reviewer is not available and a sub-reviewer will be selected.
 - 3. Consultment will be recorded within Github as a task is reviewed, with how small the group is information sharing should be streamlined (i.e. all communication will be done within a shared group chat with all members).

b. Way of working

- i. We would use Scrum-style sprints with a task board. Sprints fit our three-sprint schedule, and the board shows who is working on what. We would pair on hard tasks because we are new to Discord bots. We would not use Waterfall, because we do not yet know if the store data sources will work, so our plan may change.

c. Assigning tasks

- i. Assigning tasks will be something we do as a group. Essentially we will divide the work 4 ways and ensure each person is comfortable and confident in their ability to complete the amount of work they have been given for the sprint

d. Meetings

- i. Will meet in person 3 times a week in class
 - 1. The purpose is to discuss the project progress and next steps and complete in class assignments
 - 2. This meeting type lasts 1 hour and 15 minutes
- ii. Will meet over a discord call 1-3 times a week or as needed
 - 1. The purpose is to discuss aspects of the project
 - 2. This meeting type lasts 30 minutes to 2 hours

e. Pair-written code

- i. If two or more people wrote a change together, does it still need a separate reviewer?
 - 1. No, two people is enough to review and approve any such changes

4. Getting a change in

a. The main branch

- i. What must always be true of the main branch?
 - 1. Must be FUNCTIONAL
- ii. Branching strategy?
 - 1. There will be a main branch that remains functional at all times (in theory)

2. Feature branches will be made for all and any features for the project (must be reviewed before being pushed to main)
- b. Time on a branch
 - i. A piece of work can join the main branch once it is approved by the approver.
 - c. Joining the main branch
 - i. History will show on a secondary branch when work is joined on the main branch. A member may rebase only when no one else is using the original commits, that is if the commits are local or when all collaborators agree to switch to the rewritten history.
 - d. Version numbers
 - i. A service's version number depends on how risky an upgrade is, with major versions signaling breaking changes, minor versions adding compatible features, and patch versions fixing bugs without altering behavior.